

DOCUMENTACION DEL PROYECTO

Sistema de Evaluación y Mejora Automática de Imágenes

Análisis de código reutilizable del proyecto

Integrantes: Gómez, María Florencia; Miño, Carla; Rodríguez
Noelia

Lenguajes: Python 3 | Librerías: OpenCV, Pillow, NumPy

Entorno: VS Code

Sistema de evaluación de imágenes

Sistema automático de evaluación de calidad de imágenes. Desarrollar un sistema que analice automáticamente la calidad de una imagen y proporcione feedback (Como: La imagen tiene buena nitidez, pero está subexpuesta y con bajo contraste) y la mejore automáticamente

1. Descripción del Problema

Al comenzar el trabajo, desarrollamos una versión del proyecto en Google Colab porque estábamos teniendo problemas para trabajar correctamente desde VS Code. Esa versión nos permitió probar distintas técnicas de procesamiento de imágenes y avanzar con la lógica principal del sistema.

Una vez resueltos los inconvenientes con el entorno de desarrollo, migramos el proyecto a VS Code. Durante ese proceso también reutilizamos ideas y partes de código desarrolladas en trabajos prácticos anteriores, especialmente aquellas relacionadas con la organización en módulos y la separación de responsabilidades.

El objetivo pasó a ser integrar todo en una única versión más ordenada, aprovechando lo mejor del proyecto realizado en Colab y adaptando componentes que ya habían sido utilizados en otros ejercicios de la materia.

Problemas concretos encontrados en el código original

- **filtros.py** repetía el mismo bloque de resolución de rutas en cada función
- **El evaluador del Colab** solo media 3 métricas (brillo, contraste, nitidez), siendo muy limitado.
- **La función mejorar() del Colab** mezclaba lógica de edición, evaluación y presentación en un solo lugar.
- **save_image en io_imagenes.py** tenía dos definiciones de la misma función — Python solo usa la segunda, pero es confuso y propenso a errores.
- **El main.py original** era un menú interactivo que no encajaba con el flujo automático que queríamos lograr.

2. Enfoque y Arquitectura

La idea principal fue organizar el proyecto en varios archivos, separando las distintas tareas del sistema. De esta forma, la carga de imágenes, la edición, la evaluación y el proceso de mejora

quedaron en módulos independientes, haciendo que el código fuera más fácil de entender y modificar.

Además, esta organización permitió reutilizar partes desarrolladas anteriormente sin tener que reescribir todo el proyecto.

Estructura final del proyecto

Archivo	Responsabilidad	De donde viene
configuracion.py	Centraliza todos los umbrales y constantes. Si cambia un valor, se toca un solo archivo.	Nuevo
utilidades.py	Lectura/escritura de imágenes y conversión entre formatos OpenCV y PIL.	Reutilizado de io_imagenes.py (VSCode)
editor.py	Jerarquía de clases para aplicar transformaciones a la imagen.	Reutilizado casi íntegro del Colab
evaluador.py	Analiza una imagen y devuelve un puntaje, una clasificación y los problemas detectados	Fusión del Colab y diagnosticar_imagen (VSCode)
mejorador.py	Ciclo iterativo que usa editor.py y evaluador.py para mejorar la imagen.	Basado en mejorar() del Colab, expandido
main.py	Punto de entrada. Coordina todo y muestra resultados.	Reescrito desde cero

Que se reutilizo y que se descarto

Código original	Decisión	Motivo
Imagen Base, Tono, Filtros, Transformaciones, Editor (Colab)	Reutilizado casi íntegro	Ya resolvía correctamente las operaciones de edición que necesitábamos
Evaluador + mejorar() (Colab)	Base sólida, Sirvió como punto de partida para la versión final.	Se separo en dos archivos distintos
diagnosticar_imagen (VSCode)	Se integra como análisis detallado	Tenía 7 métricas vs 3 del Colab
read_image / save_image (VSCode)	Reutilizado directamente	Se reutilizaron con pequeñas modificaciones para adaptarlas a la nueva estructura.
filtros.py (VSCode)	Parcialmente adaptado	Algunas funciones tuvieron que modificarse para trabajar con imágenes cargadas en memoria en lugar de depender únicamente de archivos.
main.py (VSCode)	Reescrito	Era CLI interactivo; el nuevo tiene flujo automático

3. Técnicas y Conceptos Técnicos

3.1. Programación Orientada a Objetos (POO)

El módulo editor.py usa una jerarquía de herencia múltiple donde cada clase agrupa operaciones relacionadas:

- ImagenBase: encapsula la imagen PIL y expone get_image() y save().
- Tono (hereda de ImagenBase): brillo, contraste, saturación, escala de grises, ecualización de histograma.
- Filtros (hereda de ImagenBase): desenfoque, gaussiano, nitidez, bordes, relieve, reducción de ruido bilateral.
- Transformaciones (hereda de ImagenBase): rotar, redimensionar, recortar, voltear.
- Segmentación (hereda de ImagenBase): umbral, detección de contornos con Canny.
- **Editor (hereda de Tono + Filtros + Transformaciones + Segmentación):** clase principal que tiene todo disponible.

Se utilizó herencia múltiple para reunir en una misma clase las distintas operaciones de edición de imágenes. De esta manera, la clase Editor puede acceder directamente a funcionalidades relacionadas con tono, filtros, transformaciones y segmentación.

Además, cada método devuelve self, lo que permite encadenar operaciones de forma sencilla, por ejemplo:

```
editor.brillo(1.2).contraste(1.3).nitidez()
```

3.2. Métricas de Análisis de Imagen

El módulo evaluador.py mide 5 propiedades de la imagen para determinar su calidad:

Métrica	Como se mide	Librería
Brillo	Promedio de intensidad de píxeles en escala de grises (0=negro, 255=blanco)	PIL - ImageStat
Contraste	Desviación estándar de los píxeles en grises. Bajo = imagen plana sin diferencias tonales	PIL - ImageStat
Nitidez	Varianza del operador Laplaciano. Detecta cambios bruscos (bordes). Bajo = desenfocada	OpenCV - Laplacian

Ruido	Desviación estándar en una zona central de la imagen (zona uniforme). Alta = ruido visible	NumPy - std
Saturación	Promedio del canal S del espacio HSV. 0=gris puro, 255=color puro	OpenCV - cvtColor

3.3. Score Ponderado

Cada métrica produce un sub-score entre 0.0 y 1.0 según que tan lejos esta del rango ideal. El score final es un promedio ponderado:

Se decidió dar mayor peso al brillo y la nitidez porque suelen tener un impacto más evidente en la percepción general de la imagen.

El score se convierte en clasificación textual:

- 0.85 o más: Excelente
- 0.65 a 0.84: Buena
- 0.45 a 0.64: Regular
- Menos de 0.45: Mala

3.4. Conversión BGR / RGB

Una de las decisiones técnicas más importantes del proyecto. OpenCV lee imágenes en formato BGR (Azul, Verde, Rojo) mientras que PIL las espera en RGB (Rojo, Verde, Azul). Sin conversión, los colores salen invertidos: los rojos aparecen azules.

SOLUCION: Las funciones `cv_a_pil()` y `pil_a_cv()` en `utilidades.py` centralizan todas las conversiones. Ningún otro modulo tiene que preocuparse por esto.

Tambien se usó `cv2.imdecode()` en lugar de `cv2.imread()` porque `imread` falla silenciosamente con rutas que tienen tildes, espacios o caracteres especiales en Windows.

3.5. Ciclo Iterativo de Mejora

El `mejorador.py` implementa un bucle que repite hasta 5 veces el proceso de evaluar y mejorar, con dos condiciones de salida anticipada:

1. Si el score llega al mínimo aceptable (0.65), termina antes de llegar al límite de iteraciones.
2. Si una mejora empeora la imagen (score baja), restaura la versión anterior y termina.

Las mejoras se aplican siguiendo un orden específico para evitar que una operación afecte negativamente a las siguientes:

Orden	Operación	Por que este orden
1ro	Reducción de ruido	El ruido amplifica cualquier operación posterior, especialmente nitidez
2do	Ajuste de brillo	La exposición global afecta como se percibe el contraste
3ro	Ajuste de contraste	Distribución tonal sobre brillo ya corregido
4to	Saturación	Color independiente de los demás ajustes
5to	Nitidez	Siempre al final: realza bordes sin amplificar ruido ya reducido

4. Implementación: Decisión por Decisión

4.1. configuracion.py

Fue el primer archivo en implementarse porque todos los demas lo importan. Centraliza:

- Umbrales de evaluación (BRILLO_MIN, BRILLO_MAX, CONTRASTE_MIN, NITIDEZ_MIN, RUIDO_MAX, SATURACION_MIN)
- Límites de clasificación (SCORE_EXCELENTE, SCORE_BUENA, SCORE_REGULAR)
- Score mínimo aceptable para que el ciclo de mejora se detenga
- Máximo de iteraciones (MAX_ITERACIONES = 5) para evitar bucles infinitos
- Pesos de cada métrica para el score ponderado
- Rutas de entrada y salida de imágenes

Mantener estos valores en un único archivo facilita los ajustes y evita tener que modificar varios módulos cuando se desea cambiar algún criterio de evaluación.

4.2. utilidades.py — I/O y conversiones

Este módulo reúne funciones auxiliares utilizadas por distintas partes del proyecto, principalmente para la carga, guardado y conversión de imágenes.

- leer_imagen_cv(): usa imdecode para robustez con rutas especiales
- leer_imagen_pil(): carga directamente en PIL para operaciones de tono
- cv_a_pil() / pil_a_cv(): centralizan la conversión BGR/RGB
- guardar_imagen(): acepta tanto arrays OpenCV como objetos PIL Image
- imprimir_separador() / imprimir_resultado_evaluacion(): salida prolija en consola

4.3. editor.py — Clases de edición

Reutilizado casi íntegro del código Colab. La modificación principal fue en el `__init__` de `ImagenBase`:

ANTES: `def __init__(self, url: str)` — solo aceptaba rutas a archivos en disco. DESPUES: `def __init__(self, fuente)` — acepta tanto rutas (`str/Path`) como objetos `PIL Image` ya cargados en memoria. Motivo: el mejorador trabaja con imágenes en memoria. Sin este cambio, habría que guardar la imagen en disco y volver a leerla en cada iteración — lo que generaría operaciones de lectura y escritura innecesarias.

Se agregaron dos métodos que el mejorador necesitaba: `reducir_ruido()` (filtro bilateral vía `OpenCV`) y `aumentar_saturacion()` (vía `ImageEnhance`).

4.4. evaluador.py

Este módulo fusiona el Evaluador del Colab (3 métricas, score normalizado) con el `diagnosticar_imagen` del `VSCoDe` (7 métricas detalladas). El resultado final tiene 5 métricas con score ponderado.

Cada método devuelve el valor calculado, un puntaje normalizado y una descripción del problema detectado cuando corresponde:

Entrada: versión de la imagen que necesita ese análisis Salida: (valor numérico, `sub_score_0_a_1`, `descripcion_del_problema_o_None`) Si `sub_score = 1.0`, esa métrica está perfecta. Si `sub_score = 0.0`, está en su peor estado.

El evaluador no imprime nada por sí solo: devuelve un diccionario estructurado con score, clasificación, problemas y métricas para que `main.py` decida como mostrarlo.

4.5. mejorador.py

Coordina el ciclo iterativo sin evaluar ni editar directamente. Lógica de mejora diferenciada según severidad:

- Ruido alto: filtro bilateral (preserva bordes). Ruido moderado: gaussiano suave.
- Imagen muy oscura (brillo < 40): aumento de brillo fuerte (x1.5). Levemente oscura: moderado (x1.25).
- Imagen muy sobreexpuesta (brillo > 220): reducción fuerte (x0.65). Levemente: suave (x0.82).
- Contraste muy bajo (std < 15): ecualización de histograma. Bajo: aumento moderado (x1.4).
- Imagen muy desenfocada: doble pasada de nitidez. Levemente: pasada simple.
- Sin problemas específicos, pero score bajo: mejora genérica.

4.6. main.py

La función principal de este módulo es coordinar la ejecución del sistema. Recibe la imagen indicada por el usuario, ejecuta la evaluación correspondiente y, si es necesario, inicia el proceso de mejora

```
python main.py img_input/foto.jpg
python main.py img_input/foto.jpg --salida resultado.jpg
```

La decisión de iniciar o no el ciclo de mejora depende del puntaje obtenido y de los problemas detectados durante la evaluación:

- Score bueno + sin problemas: guarda y termina sin iterar.
- Score bueno + con problemas: avisa y de todas formas entra al ciclo de mejora.
- Score bajo: entra al ciclo de mejora directamente.

5. Resultados Obtenidos

5.1. Salida del sistema

Al ejecutar el sistema, la consola muestra un informe estructurado en etapas:

Sección del informe	Que muestra
EVALUACION INICIAL	Score (0.0 a 1.0 y porcentaje), clasificación, lista de problemas y 5 métricas numéricas

INICIANDO CICLO DE MEJORA	Score mínimo requerido y máximo de iteraciones configurado
Iteración N	Nuevas métricas, mejoras aplicadas, si hubo restauración por empeoramiento
RESUMEN FINAL	Score inicial vs final, mejora porcentual, total de iteraciones, si se alcanzó el mínimo

5.2. Archivos generados

- `img_output/[nombre]_original.jpg`: copia de la imagen de entrada antes de cualquier procesamiento.
- `img_output/imagen_mejorada.jpg` (o el nombre que se indique): imagen final procesada.

5.3. Ejemplo real de prueba

Se probó con una imagen de un departamento (736x1308 px):

Métrica	Valor inicial	Estado
Brillo	65.8 / 255	OK (rango 60-190)
Contraste	51.5 std	OK (mínimo 35)
Nitidez	203.4	OK (mínimo 80)
Ruido	21.7 std	PROBLEMA (máximo 20)
Saturación	94.5 / 255	OK (mínimo 25)
Score total	0.99 (98.7%)	Excelente

Aunque la imagen ya presentaba una calidad general alta, el sistema detectó un nivel de ruido ligeramente superior al umbral configurado. Debido a esto, se ejecutó igualmente el proceso de mejora para intentar reducir dicho problema sin afectar el resto de las métricas.

Este caso permitió comprobar que el sistema no se basa únicamente en el puntaje global, sino también en los problemas específicos detectados durante la evaluación.

6. Pruebas Realizadas

6.1. Bug encontrado y corregido durante las pruebas

Al probar con una imagen(score 0.85, 1 problema de ruido):

- El sistema mostro el mensaje 'Se intentará mejorar de todas formas' (main.py funcionaba bien)...
- ...pero el RESUMEN FINAL decía '0 iteraciones' y 'Mejora total: +0.0%'.

Durante las pruebas se detectó una inconsistencia entre la lógica utilizada por main.py y mejorador.py para decidir cuándo iniciar el proceso de mejora. En algunos casos el sistema indicaba que intentaría corregir la imagen, pero el ciclo de mejora no llegaba a ejecutarse.

El problema fue corregido unificando las condiciones utilizadas por ambos módulos.

Solución aplicada en mejorador.py:

```
# ANTES
if score_inicial >= SCORE_MINIMO_ACEPTABLE:
    return ... # terminaba sin iterar

# DESPUES
hay_problemas = len(evaluacion_inicial['problemas']) > 0
if score_inicial >= SCORE_MINIMO_ACEPTABLE and not hay_problemas:
    return ... # solo termina si score bueno Y sin problemas
```

6.2. Limitaciones encontradas

Durante las pruebas se observó que algunas métricas dependen del tipo de imagen analizada. Por ejemplo, fotografías con muchas texturas o detalles pueden ser interpretadas como más ruidosas de lo que realmente son.

Además, los valores de referencia utilizados para brillo, contraste y saturación fueron ajustados a partir de las imágenes de prueba disponibles, por lo que podrían requerir modificaciones para otros conjuntos de imágenes.

7. Posibles Mejoras Futuras

7.1. Mejoras al sistema de evaluación

- Agregar análisis de distribución tonal (histograma): detectar clipping (áreas totalmente blancas o negras).

- Implementar detección de bordes más sofisticada para imágenes con texturas muy complejas.
- Agregar comparación antes/después para calcular mejora objetiva por métrica individual.

7.2. Mejoras al ciclo de mejora

- Implementar CLAHE (ecualización de histograma adaptativa por sectores) para contraste local.
- Agregar ajuste adaptativo: si después de N iteraciones el score no mejora significativamente, detener el ciclo.
- Permitir configurar desde la terminal que métricas priorizar (ejemplo: --priorizar nitidez).

7.3. Mejoras de arquitectura

- Incorporar el algoritmo BRISQUE para obtener una evaluación de calidad de imagen más avanzada.
- Generar un informe visual con las métricas obtenidas antes y después del procesamiento.
- Agregar nuevos filtros y técnicas de mejora para trabajar con imágenes que presenten problemas más complejos.

8. Funcionamiento General del Proyecto

El funcionamiento del sistema comienza cuando el usuario indica la imagen que desea procesar desde la terminal. A partir de esa imagen, el módulo evaluador analiza distintas características como brillo, contraste, nitidez, ruido y saturación.

Con los resultados obtenidos, el sistema determina si la imagen necesita correcciones. En caso de detectar problemas o de obtener un puntaje bajo, se inicia el proceso de mejora automática.

Durante este proceso, se aplican distintas operaciones de edición según los problemas encontrados. Luego de cada modificación, la imagen vuelve a evaluarse para comprobar si la calidad mejoró.

El ciclo continúa hasta alcanzar una calidad aceptable o hasta llegar al número máximo de iteraciones configurado. Finalmente, la imagen resultante se guarda en la carpeta de salida y se muestra un resumen con los resultados obtenidos.
